UNIT 12 75 min

Advanced Functions and Comprehensions

Flexible arguments, lambda, map/filter/zip, comprehensions, sorting with key, and recursion.

By the end of this unit you will be able to

- Write a function that accepts any number of positional and keyword arguments
- Use `lambda` for a short, unnamed function
- Transform and select data with `map`, `filter`, and `zip`
- Write list, dictionary, and set comprehensions, and decide when a plain loop is clearer
- Sort with a `key` function
- Write a recursive function with a base case

Flexible arguments: *args and **kwargs

`*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dictionary. The names `args` and `kwargs` are conventions; the `*` and `**` are what matter.

flexible.py

```
def total(*numbers):
    return sum(numbers)

def describe(**details):
    for key, value in details.items():
        print(f"{key}: {value}")

print(total(1, 2, 3, 4))
describe(name="Ada", course="Python")
```

lambda: a small unnamed function

`lambda parameters: expression` makes a function with no name. It can hold one expression only. Use it when you need a short function to hand to another function, such as a sort key. Name the

function with `def` when it needs more than one line.

lambda.py

```
square = lambda x: x * x
print(square(7))

pairs = [(1, "pear"), (3, "apple"), (2, "fig")]
print(sorted(pairs))                       # sorts by the first item
print(sorted(pairs, key=lambda p: p[1]))   # sorts by the name
```

map, filter, and zip

`map(func, items)` applies a function to each item. `filter(func, items)` keeps items for which the function returns True. `zip(a, b)` pairs items from two sequences. In modern Python, `map`, `filter`, and `zip` return iterators, so wrap them in `list()` to see the results.

map_filter_zip.py

```
prices = [2.5, 4.0, 1.25]
print(list(map(lambda p: p * 1.07, prices)))
print(list(filter(lambda p: p > 2, prices)))

names = ["Ana", "Ben"]
scores = [91, 84]
print(list(zip(names, scores)))
for name, score in zip(names, scores):
    print(name, score)
```

Comprehensions

A **comprehension** builds a collection in one expression. The pattern is `[expression for item in sequence if condition]`. Use the same idea with `{}` for a dictionary or set.

comprehensions.py

```
nums = range(1, 11)

squares = [n * n for n in nums]
evens = [n for n in nums if n % 2 == 0]
labels = ["even" if n % 2 == 0 else "odd" for n in range(4)]
lengths = {word: len(word) for word in ["cat", "horse"]}
unique_lengths = {len(word) for word in ["cat", "dog", "horse"]}

print(squares)
print(evens)
print(labels)
print(lengths)
print(unique_lengths)
```

> **Tip**
> If a comprehension needs more than one loop or a long condition, write a normal loop. Clear code beats short code. Your reader is the priority.

A **generator expression** uses parentheses instead of brackets. It produces values one at a time, which saves memory on large data: `sum(n * n for n in range(1000))`.

Sorting with key

`sorted()` and `list.sort()` accept a `key` function. Python sorts by the value that function returns, not by the item itself. Add `reverse=True` for descending order.

sort_key.py

```
students = [
    {"name": "Ana", "score": 88},
    {"name": "Ben", "score": 95},
    {"name": "Cy", "score": 72}
]
ranked = sorted(students, key=lambda s: s["score"], reverse=True)
for s in ranked:
    print(s["name"], s["score"])
```

Recursion

A **recursive** function calls itself on a smaller version of the problem. Every recursive function needs a **base case** that stops the calls, and each call must move toward it. Without a base case you get a `RecursionError`.

Trace: factorial(3) TRACE

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(3))
```

Step	Line run	Variables after the line	Output so far
1	1	`factorial = function`	—
2	2	`factorial = function, n = 3`	—
3	4	`factorial = function, n = 3`	—
4	2	`factorial = function, n = 2`	—
5	4	`factorial = function, n = 2`	—
6	2	`factorial = function, n = 1`	—
7	3	`factorial = function, n = 1`	—
8	4	`factorial = function, n = 2`	—
9	4	`factorial = function, n = 3`	—
10	6	`factorial = function`	`6`

Predict the output THINK FIRST

Watch for lazy iterators and for lambdas that return values.

```
def f(*args):
    return len(args)

print(f(1, 2, 3))
```

Answer: `3`

```
print(list(map(lambda x: x + 1, [1, 2])))
```

Answer: `[2, 3]`

```
print(list(filter(lambda x: x > 2, [1, 2, 3, 4])))
```

Answer: `[3, 4]`

```
print(list(zip([1, 2], "ab")))
```

Answer: `[(1, 'a'), (2, 'b')]`

```
print([n for n in range(6) if n % 3 == 0])
```

Answer: `[0, 3]`

```
print(sorted(["bb", "a", "ccc"], key=len))
```

Answer: `['a', 'bb', 'ccc']`

```
print(sum(n for n in range(4)))
```

Answer: `6`

Practice

▼ **Exercise 1 — Average of anything**

Write `average(*values)` that returns the average of any number of arguments. Test it with `average(2, 4, 9)`.

▼ **Exercise 2 — Recursive sum**

Write `total(items)` that returns the sum of a list using recursion only (no `sum()`, no loops). Start with the base case of an empty list.

▼ **Hint**

The sum of a list is its first item plus the sum of the rest: `items[1:]`.

Discussion

1. When is a comprehension easier to read than a loop, and when is it harder?
2. Why does `map` return an iterator instead of a list? What does that change for large files?
3. A recursive solution and a loop solution can compute the same answer. What does each one make easier to see?

Key terms

***args**

A parameter that collects extra positional arguments into a tuple.

****kwargs**

A parameter that collects extra keyword arguments into a dictionary.

lambda

A one-expression, unnamed function, written `lambda x: x * 2`.

map

A built-in that applies a function to each item of a sequence, returning an iterator.

filter

A built-in that keeps the items for which a function returns True.

zip

A built-in that pairs up items from several sequences, stopping at the shortest.

Comprehension

A compact expression that builds a list, dictionary, or set from a loop and an optional condition.

Generator expression

A comprehension in parentheses that produces values lazily, one at a time.

Iterator

An object that produces items one at a time and can be looped over once.

Recursion

A function that calls itself on a smaller problem until it reaches a base case.

Base case

The condition that stops a recursive function from calling itself again.

See all terms in the glossary →